# CS1312
# Design and Analysis of Algorithms

## Sorting

# How fast can we sort ?

- Selection Sort, Insertion Sort: O ($n^2$)
- Heap Sort, Merge sort: O (n log n)
- Quicksort: O (n log n) [ best and average case] ,

  O ($n^2$) [worst case]
- What is common to all these algorithms?

  the sorted order they determine is based only on comparisons between the input elements.

  $a_i < a_j$, $a_i \leq a_j$, $a_i = a_j$, $a_i \geq a_j$, or $a_i > a_j$

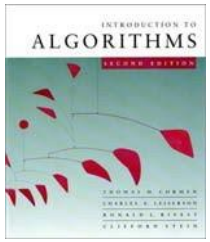
# Lower-Bound for Sorting

- **Theorem:** to sort *n* elements, comparison sorts must make $\Omega(n \log n)$ comparisons in the worst case.
- We can do better with linear sorting algorithms
  - Counting Sort
  - Radix Sort
  - Bucket sort
- Make certain assumptions about the data.
- Linear sorts are NOT “comparison sorts”.

# Counting Sort

# Counting Sort

- Counting sort assumes that each of the n input elements is an integer in the range 0 to k, for some integer k.
- It assumes that the input consists of integers in a small range.
- Counting sort determines, for each input element x, the number of elements less than x.
- It uses this information to place element x directly into its position in the output array.

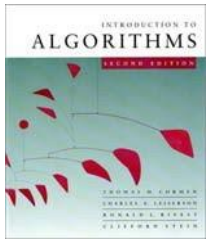



# Sorting in linear time

**Counting sort:** No comparisons between elements.

- ***Input***: $A[1 \ldots n]$, where $A[j] \in \{1, 2, \ldots, k\}$.
- ***Output***: $B[1 \ldots n]$, sorted.
- ***Auxiliary storage***: $C[1 \ldots k]$.

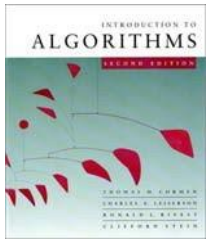

*Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson*



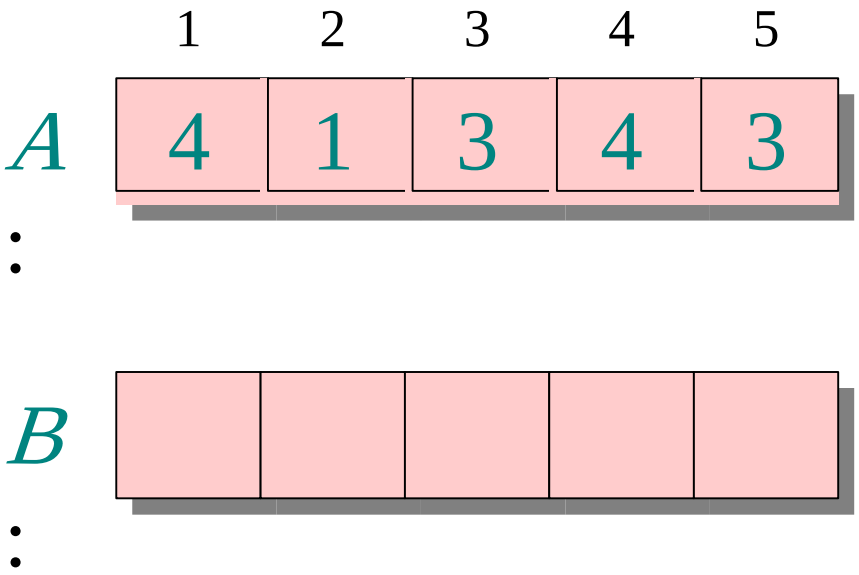

# Counting sort

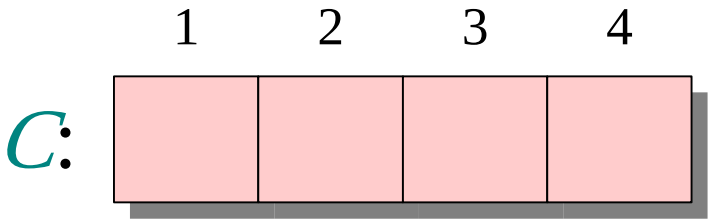
**for** $i \leftarrow 1$ **to** $k$
  **do** $C[i] \leftarrow 0$
**for** $j \leftarrow 1$ **to** $n$
  **do** $C[A[j]] \leftarrow C[A[j]] + 1$ ◁ $C[i] = |\{\text{key} = i\}|$
**for** $i \leftarrow 2$ **to** $k$ ◁ $C[i] = |\{\text{key} \le i\}|$
  **do** $C[i] \leftarrow C[i] + C[i–1]$
**for** $j \leftarrow n$ **downto** 1
  **do** $B[C[A[j]]] \leftarrow \text{A}[j]$
    $C[A[j]] \leftarrow C[A[j]] – 1$

 *Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson* 

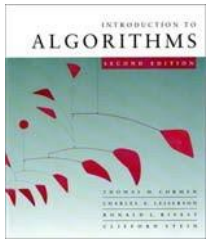



# Counting-sort example

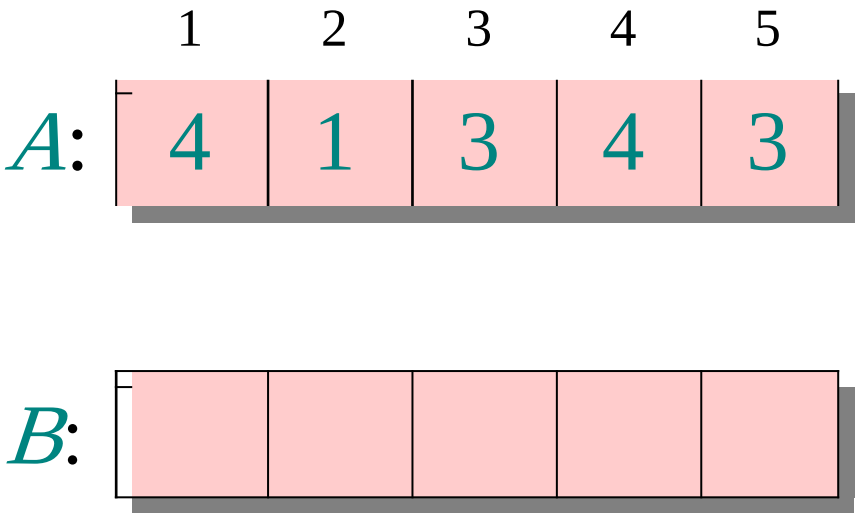


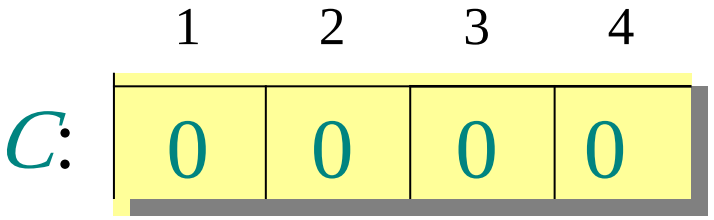




 *Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson* 



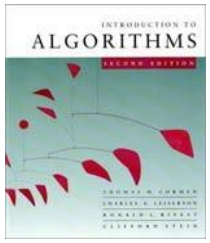

# Loop 1



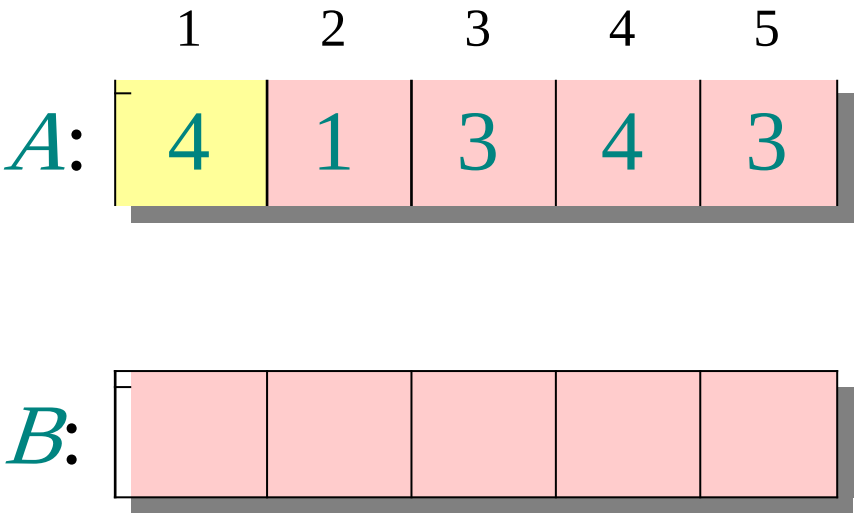



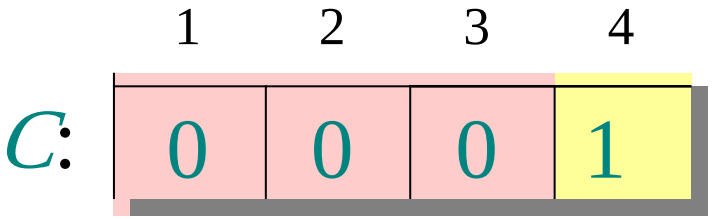

**for** $i \leftarrow 1$ **to** $k$
  **do** $C[i] \leftarrow 0$

 *Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson* 



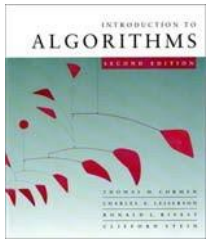
# Loop 2



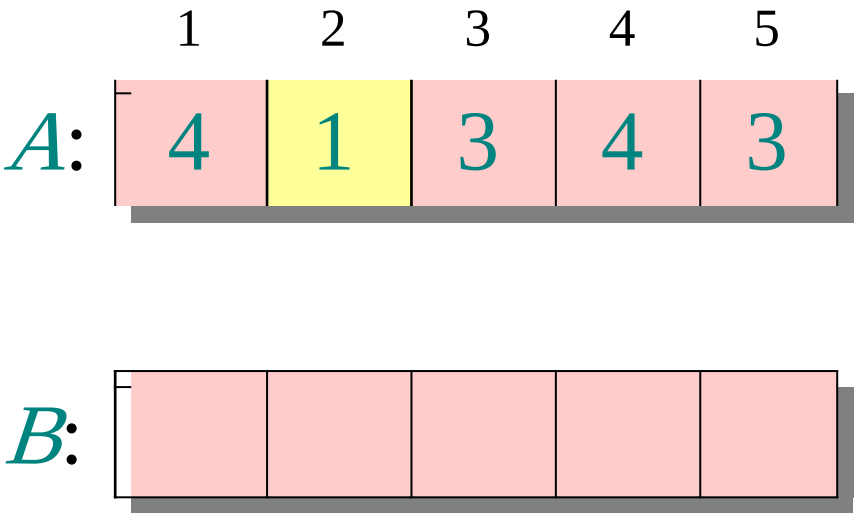



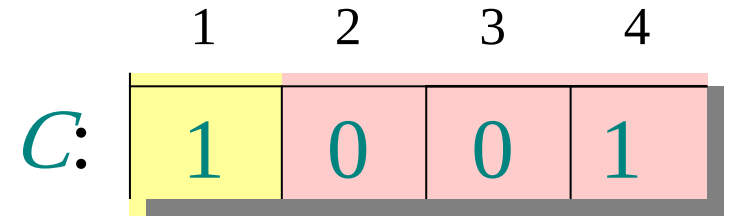

**for** $j \leftarrow 1$ **to** $n$
**do** $C[A[j]] \leftarrow C[A[j]] + 1$ ◁ $C[i] = |\{\text{key} = i\}|$


*Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson*



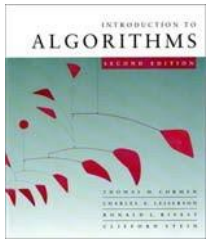

# Loop 2



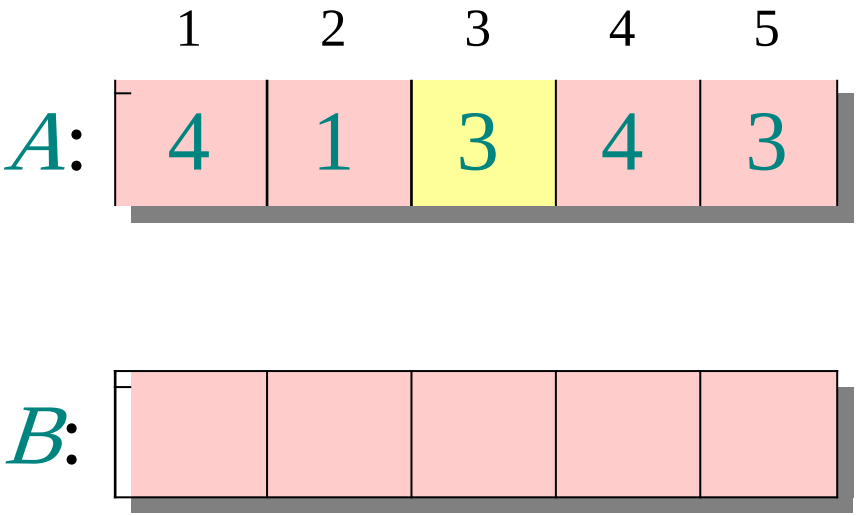



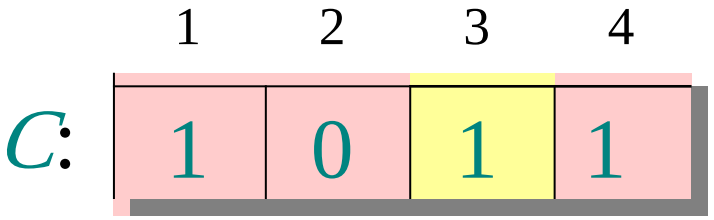

**for** $j \leftarrow 1$ **to** $n$
  **do** $C[A[j]] \leftarrow C[A[j]] + 1$ ◁ $C[i] = |\{\text{key} = i\}|$

 *Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson* 



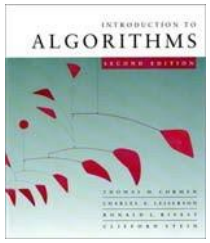

# Loop 2



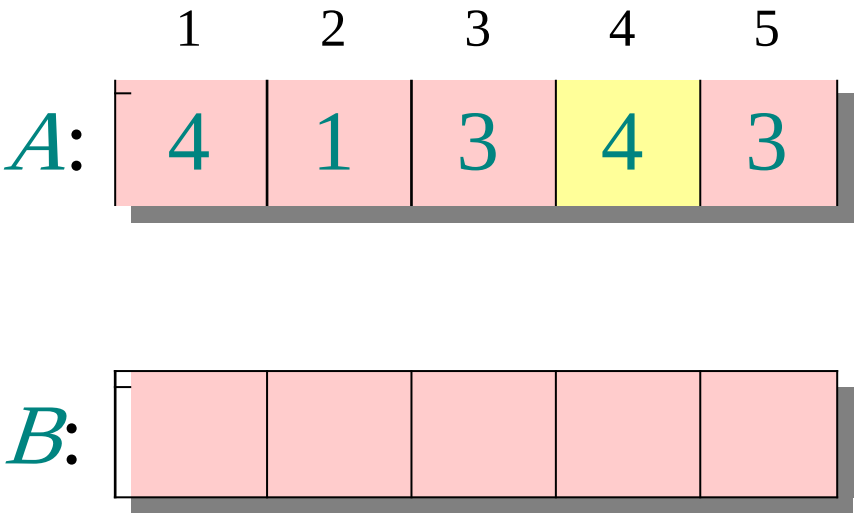



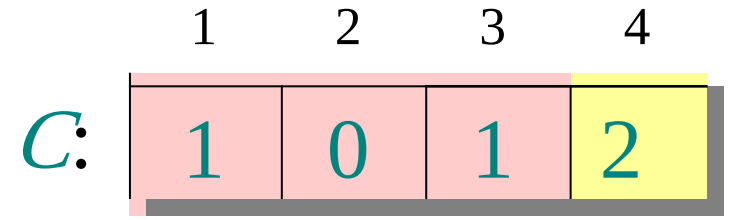

**for** $j \leftarrow 1$ **to** $n$
  **do** $C[A[j]] \leftarrow C[A[j]] + 1$ ◁ $C[i] = |\{\text{key} = i\}|$

 *Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson* 



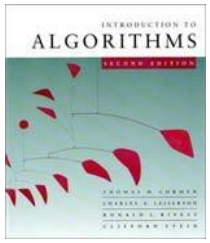

# Loop 2



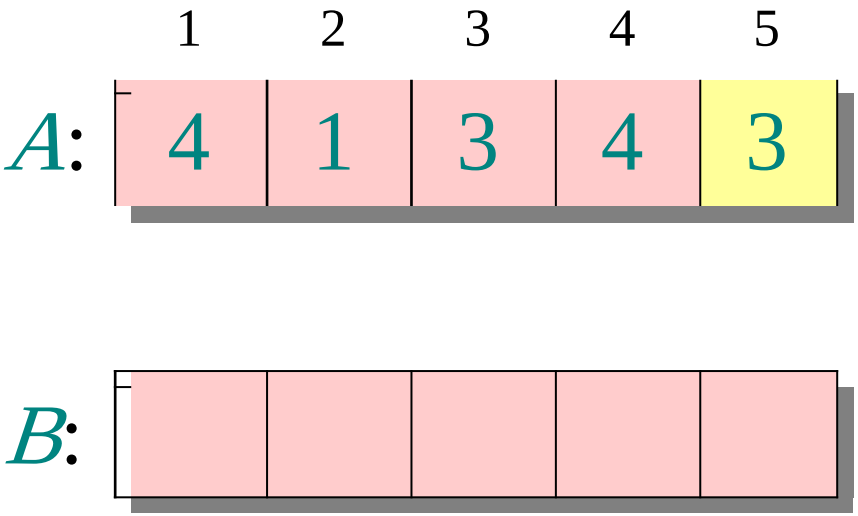



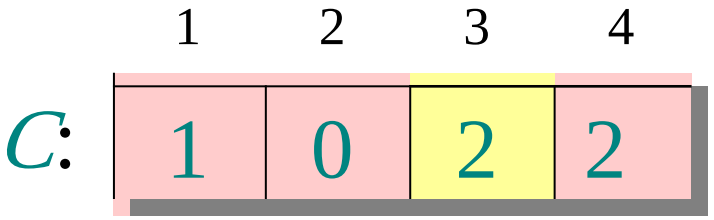

**for** $j \leftarrow 1$ **to** $n$
  **do** $C[A[j]] \leftarrow C[A[j]] + 1$ ◁ $C[i] = |\{\text{key} = i\}|$

 *Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson* 



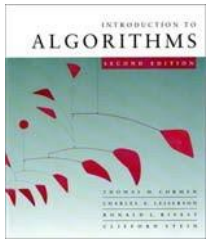

# Loop 2



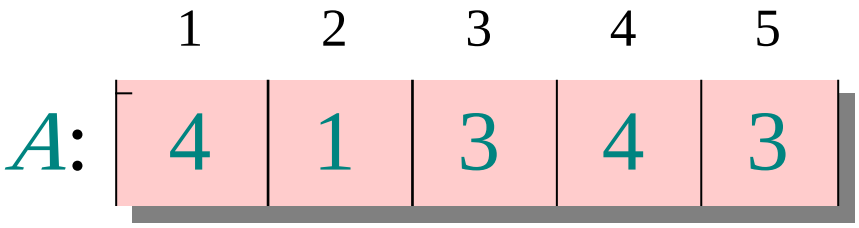



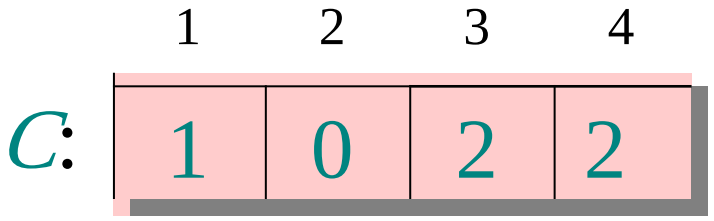

**for** $j \leftarrow 1$ **to** $n$
  **do** $C[A[j]] \leftarrow C[A[j]] + 1$ ◁ $C[i] = |\{\text{key} = i\}|$

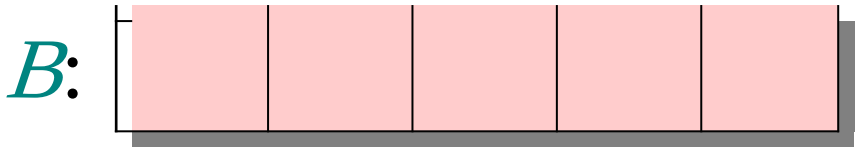
 *Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson* 

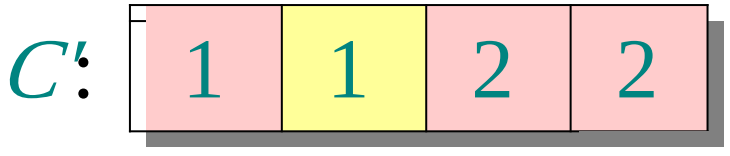



# Loop 3



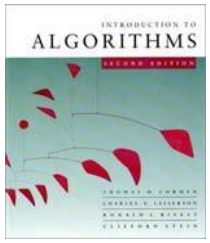



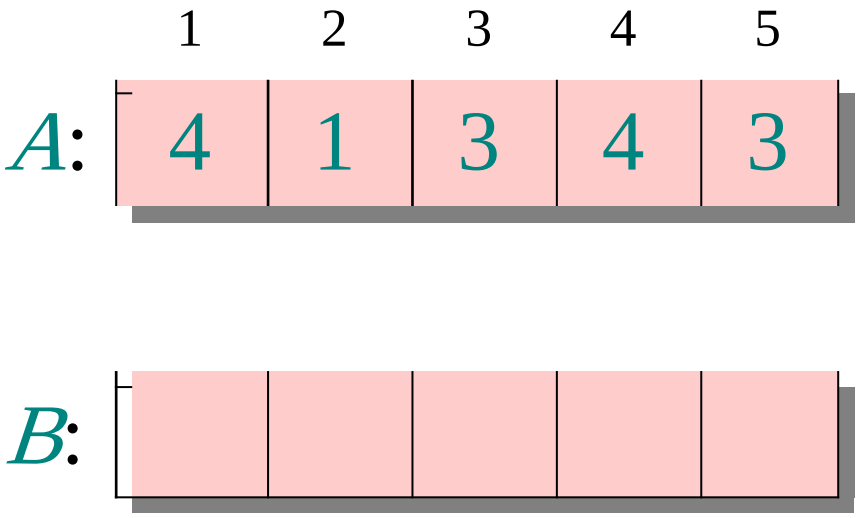

**for** $i \leftarrow 2$ **to** $k$

**do** $C[i] \leftarrow C[i] + C[i-1]$ ◁ $C[i] = |\{\text{key} \le i\}|$

 *Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson* 



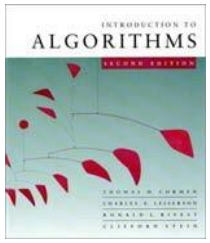

# Loop 3



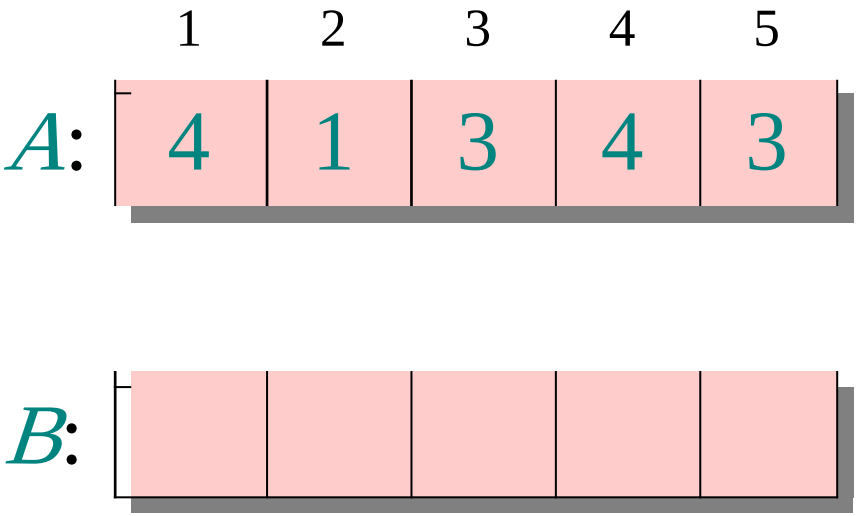

**for** $i \leftarrow 2$ **to** $k$

**do** $C[i] \leftarrow C[i] + C[i-1]$ ◁ $C[i] = |\{\text{key} \leq i\}|$

 *Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson* 



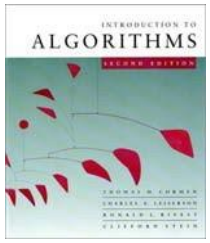

# Loop 3



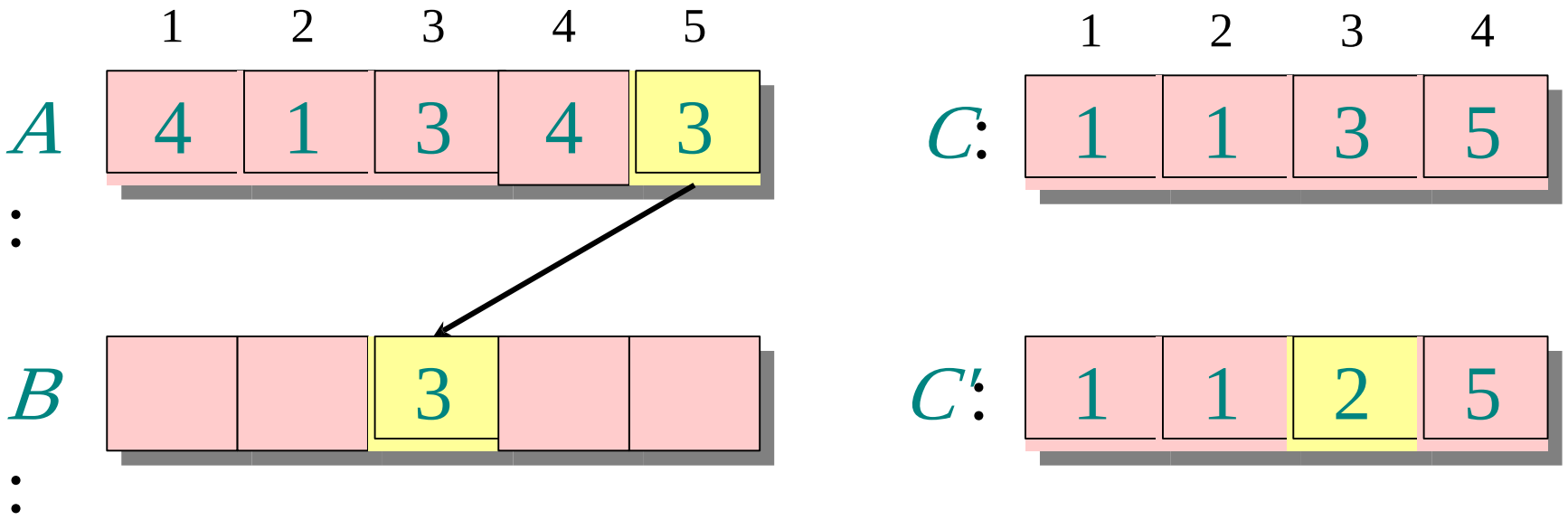

**for** $i \leftarrow 2$ **to** $k$

**do** $C[i] \leftarrow C[i] + C[i-1]$ ◁ $C[i] = |\{\text{key} \le i\}|$

 *Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson* 



# Loop 4

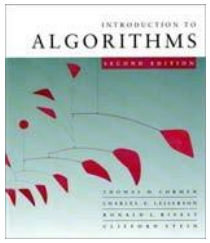



**for** $j \leftarrow n$ **downto** 1

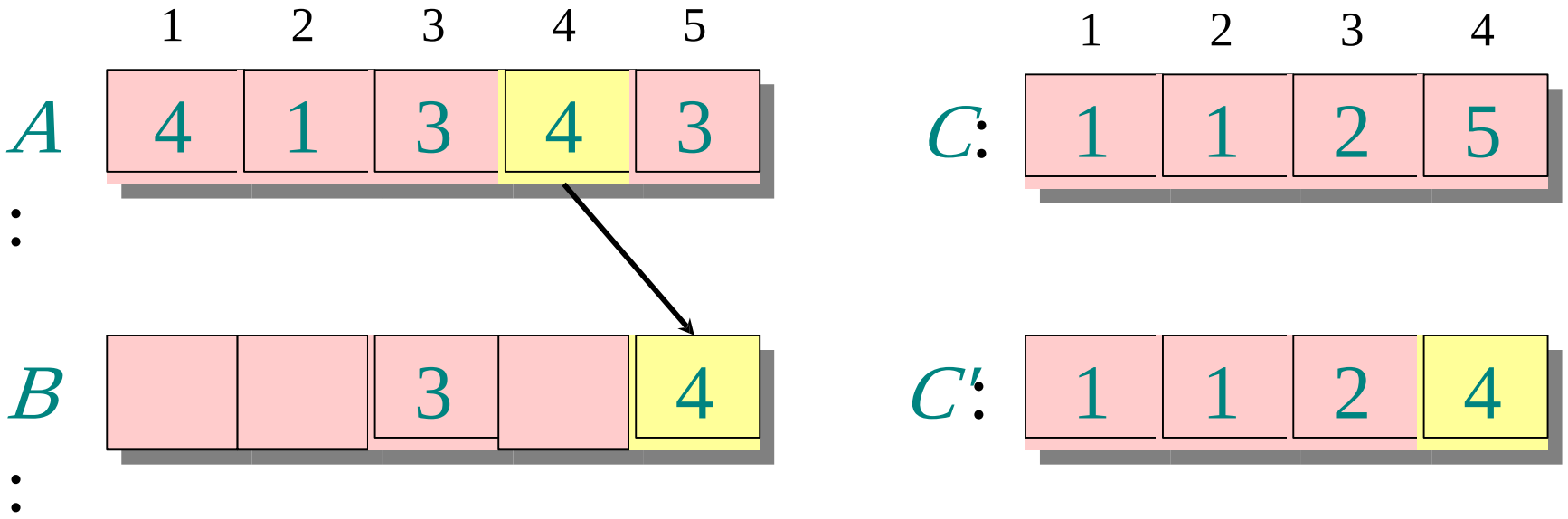
**do** $B[C[A[j]]] \leftarrow A[j]$

$C[A[j]] \leftarrow C[A[j]] - 1$


*Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson*



# Loop 4

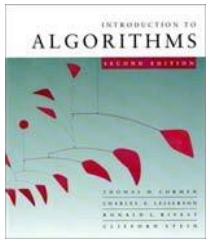



**for** $j \leftarrow n$ **downto** 1

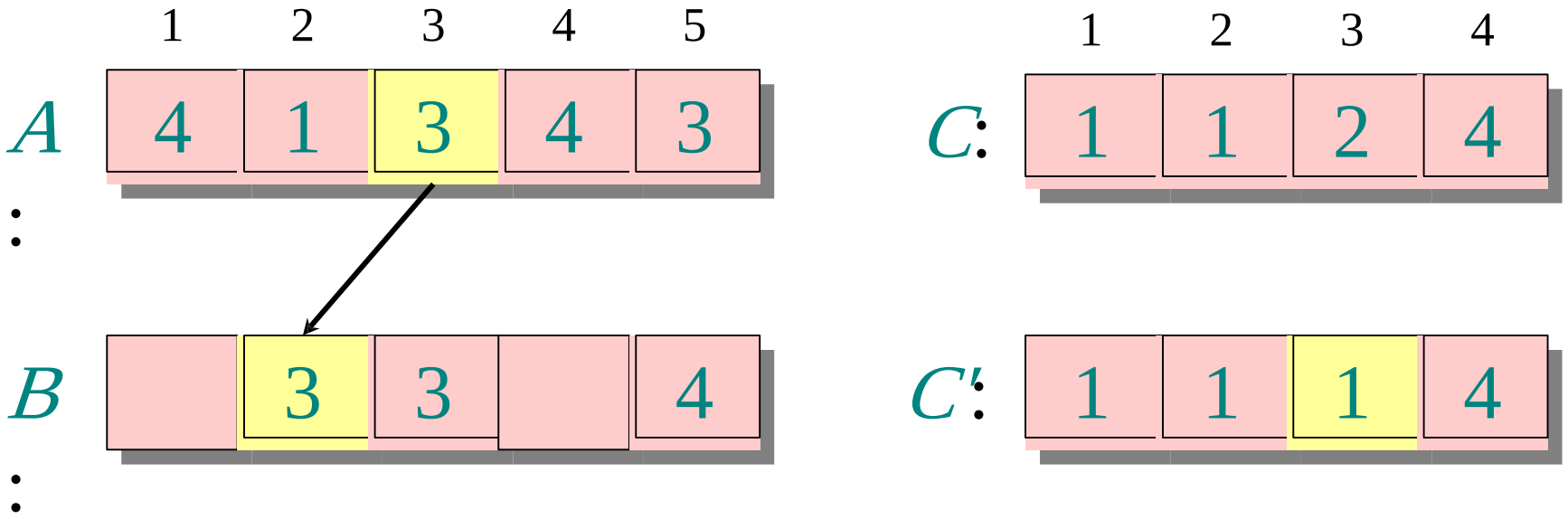
**do** $B[C[A[j]]] \leftarrow A[j]$

$C[A[j]] \leftarrow C[A[j]] - 1$


*Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson*



# Loop 4

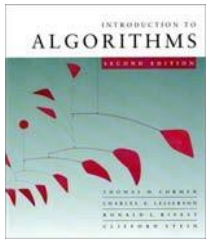



**for** $j \leftarrow n$ **downto** 1

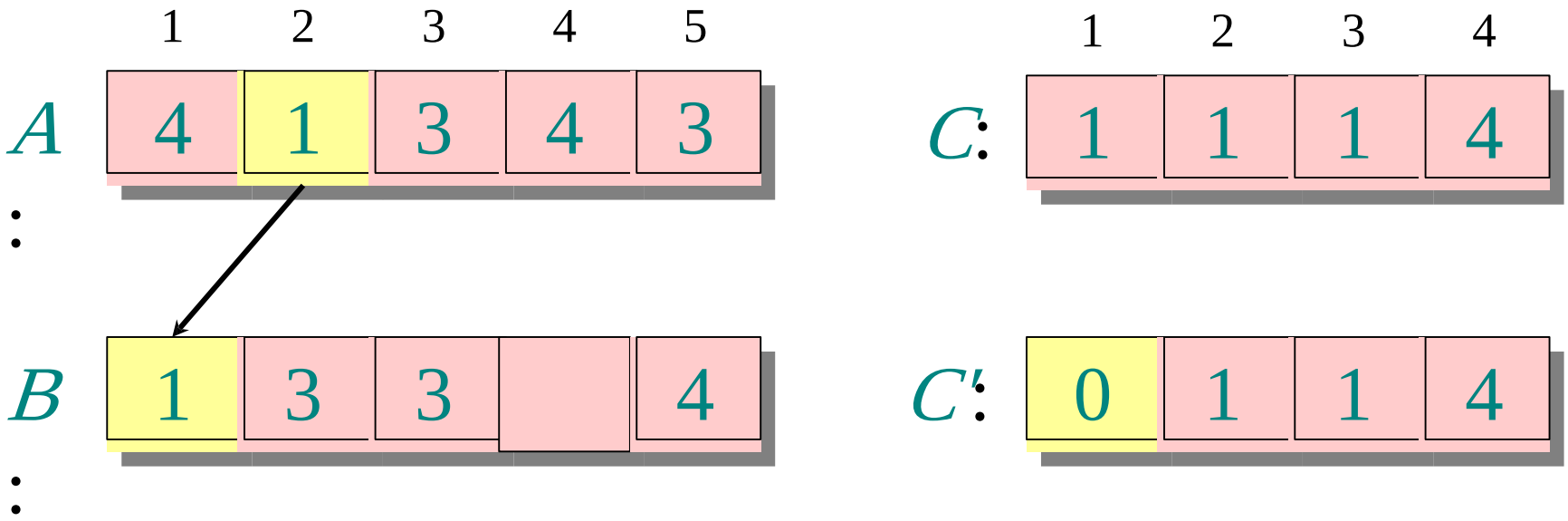
**do** $B[C[A[j]]] \leftarrow A[j]$

$C[A[j]] \leftarrow C[A[j]] - 1$


*Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson*



# Loop 4

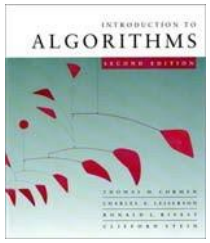



**for** $j \leftarrow n$ **downto** 1

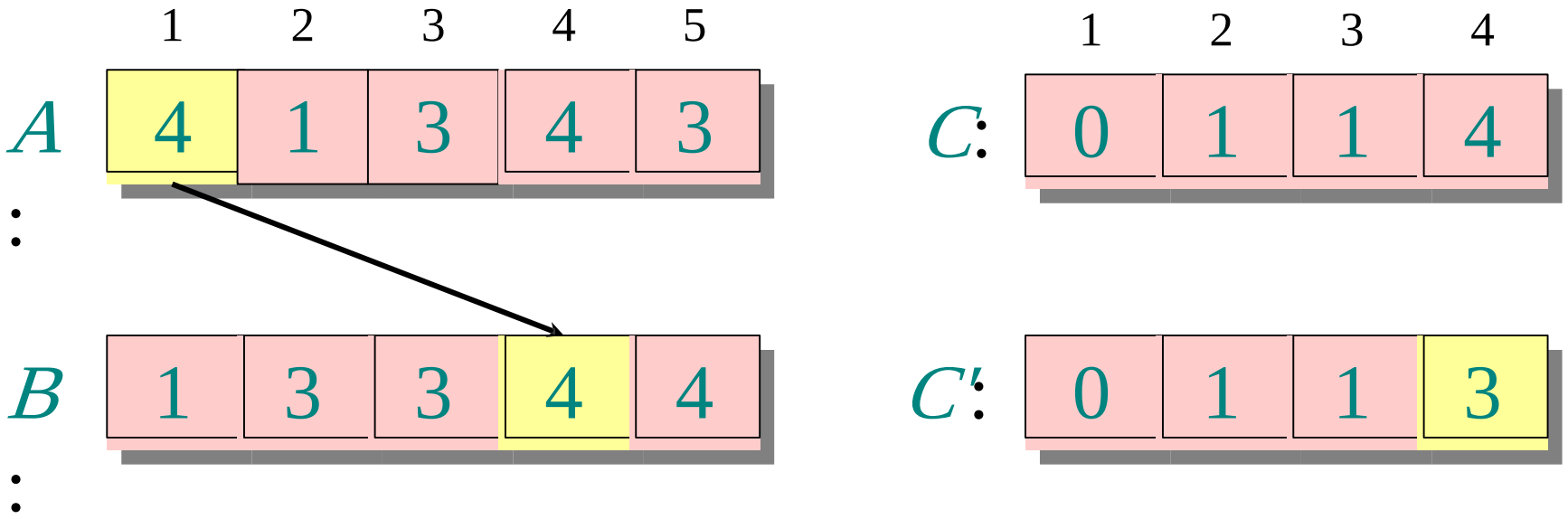
**do** $B[C[A[j]]] \leftarrow A[j]$

$C[A[j]] \leftarrow C[A[j]] - 1$


*Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson*



# Loop 4

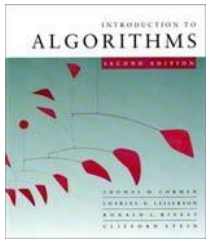



**for** $j \leftarrow n$ **downto** 1

**do** $B[C[A[j]]] \leftarrow A[j]$

$C[A[j]] \leftarrow C[A[j]] - 1$


*Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson*




# Analysis

$\Theta(k)$ **for** $i \leftarrow 1$ **to** $k$
**do** $C[i] \leftarrow 0$

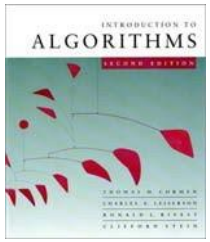
$\Theta(n)$ **for** $j \leftarrow 1$ **to** $n$
**do** $C[A[j]] \leftarrow C[A[j]] + 1$

$\Theta(k)$ **for** $i \leftarrow 2$ **to** $k$
**do** $C[i] \leftarrow C[i] + C[i-1]$

$\Theta(n)$ **for** $j \leftarrow n$ **downto** 1
**do** $B[C[A[j]]] \leftarrow A[j]$
$C[A[j]] \leftarrow C[A[j]] - 1$

$\Theta(n + k)$


*Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson*




# Running time

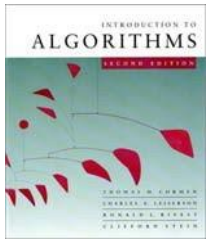
If $k = O(n)$, then counting sort takes $\Theta(n)$ time.

- But, sorting takes $\Omega(n \lg n)$ time!
- Where's the fallacy?

**Answer:**

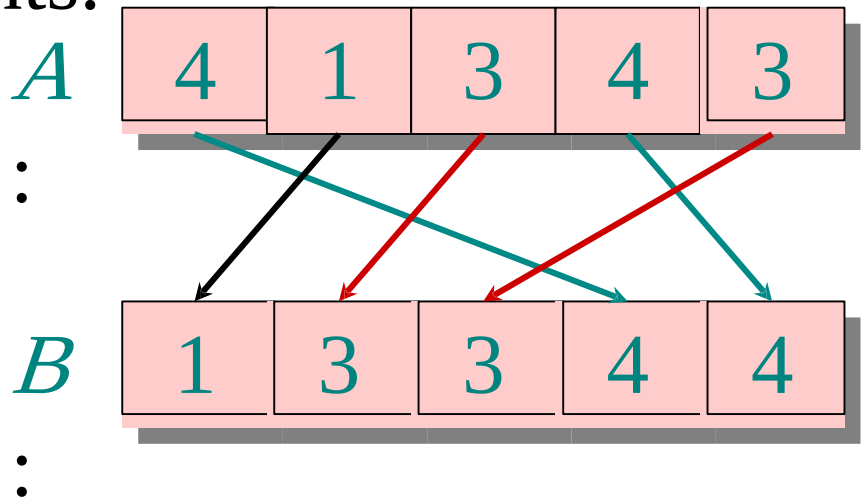
- ***Comparison sorting*** takes $\Omega(n \lg n)$ time.
- Counting sort is not a ***comparison sort***.
- In fact, not a single comparison between elements occurs!


*Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson*




# Stable sorting

Counting sort is a ***stable*** sort: it preserves the input order among equal elements.




**Exercise:** What other sorts have this property?


*Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson*

# Radix Sort

# Radix Sort

- It is a sorting algorithm that is used to sort numbers.
- The numbers are sorted from least significant digit to the most significant digit.
- If we are dealing with unsorted numbers then we know that there are 10 digits from 0 to 9 that are used to form any number

  ➔ we will need 10 bucket 0 – 9 to sort numbers

# Radix Sort

To sort numbers using Radix sort :

- Find the number of digit in the biggest number
- If there are **d** number of digit in the biggest number, it will need to perform **d** number of passes.
- The remaining numbers must be padded with zeros so they all have **d** digits
- Then we will take 10 buckets labeled from 0 – 9 .

# Radix Sort

- Sort the numbers from least significant digit.
- After the sorting is complete, remove the leading zeros.
- Radix sort algorithm :

The following procedure assumes that each element in the n-element array **A** has **d** digits

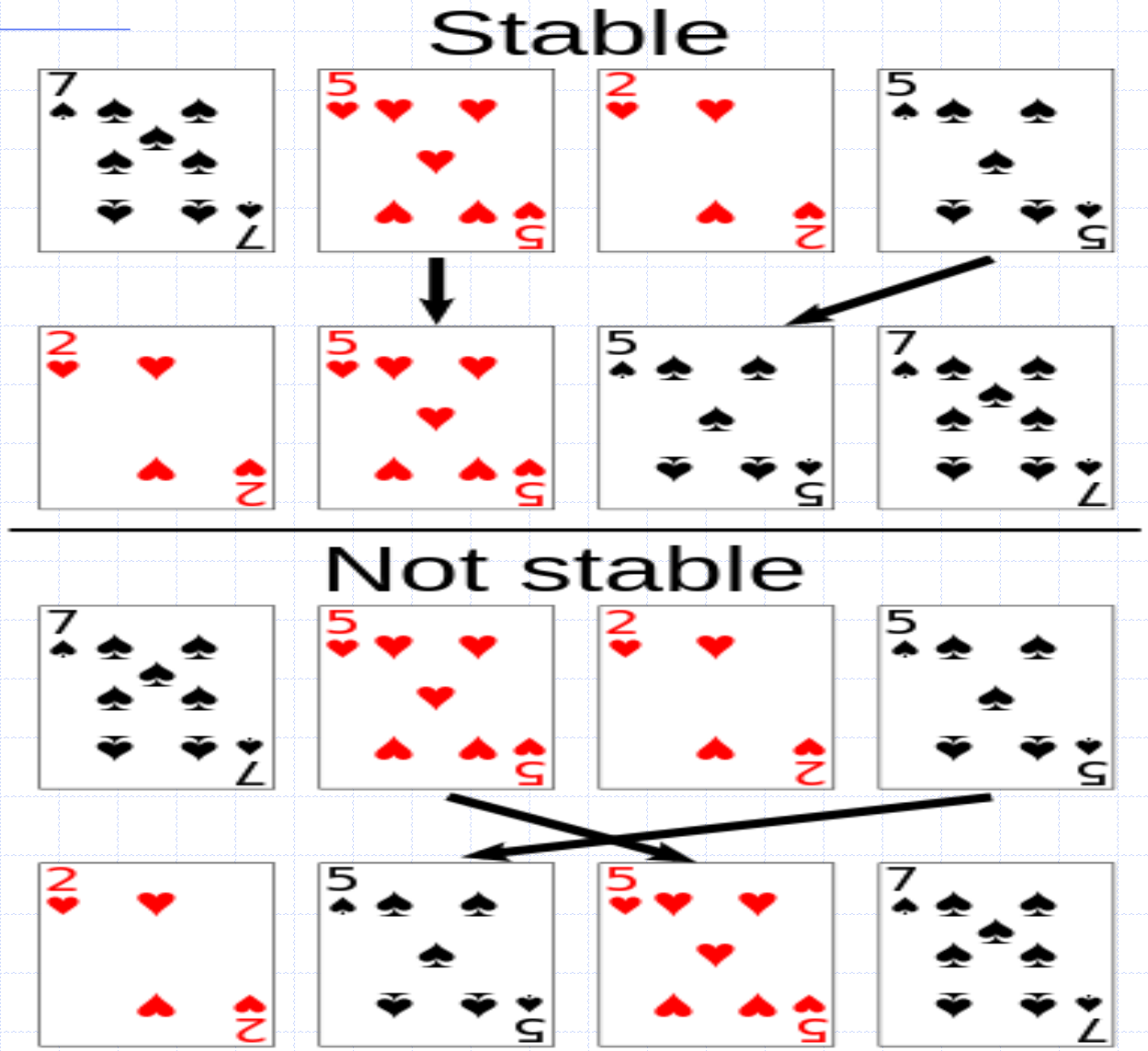
```
RADIX-SORT(A, d)
1  for i = 1 to d
2      use a stable sort to sort array A on digit i
```

# Radix Sort

What does “stable” means ?

- A sorting algorithm is said to be stable if two objects with equal keys appear in the same order in sorted output as they appear in the input array to be sorted.
- Insertion sort, Merge Sort, Bubble Sort are stable.
- Heap Sort, Quick Sort are not stable

# Radix Sort

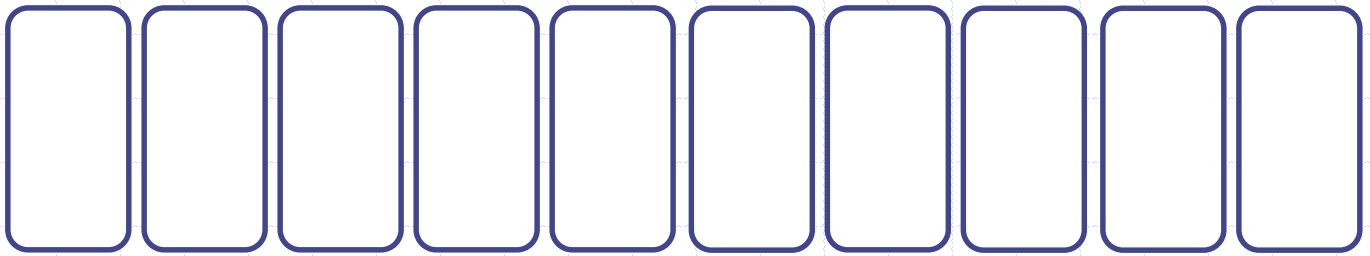


# Radix Sort – example

**10, 15, 1, 60, 5, 100 , 25, 50**

- The biggest number of digits is 3 in 100
- We will do 3 passes to sort numbers
- Pad the remaining numbers:

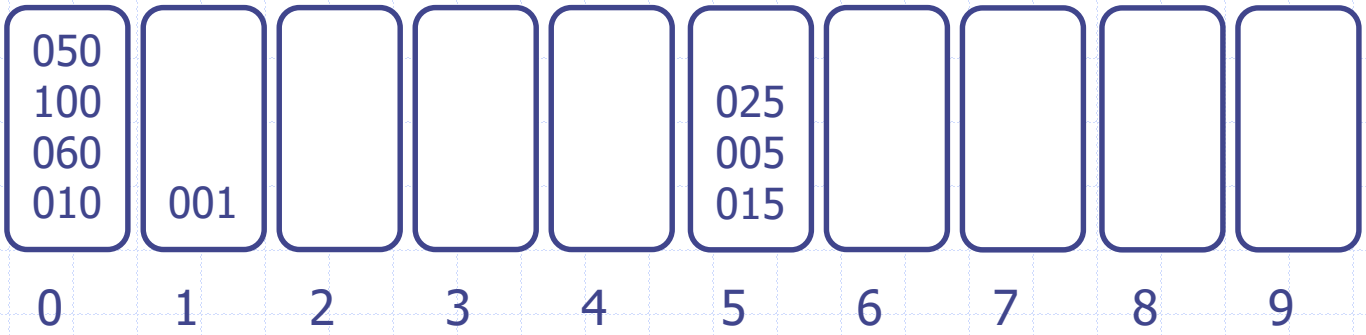
  010, 015, 001, 060 , 005, 100, 025, 050
- Use 10 bucket labelled : 0 – 9



# Radix Sort – example

**010, 015, 001, 060, 005, 100 , 025, 050**

- Sort numbers using the first digit :



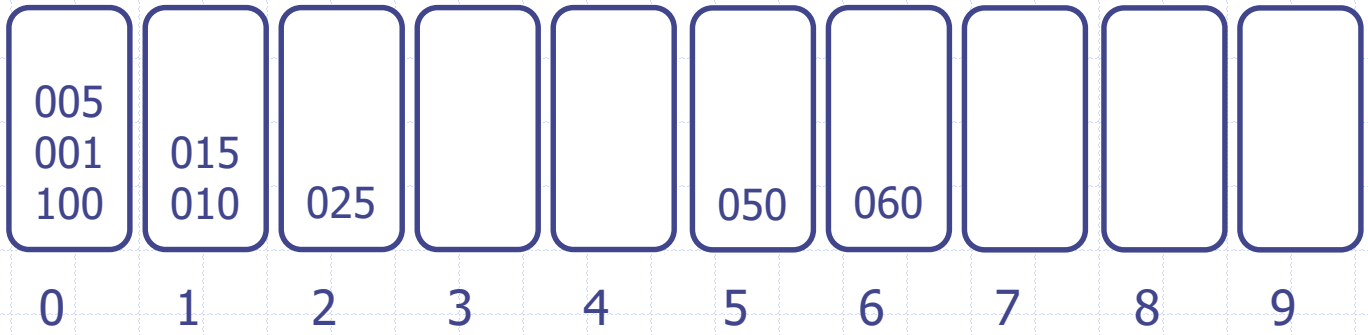

**010, 060, 100, 050, 001, 015, 005, 025**

Pass 1 is completed

# Radix Sort – example

**010, 060, 100, 050, 001, 015, 005, 025**

- Sort numbers using the second digit :



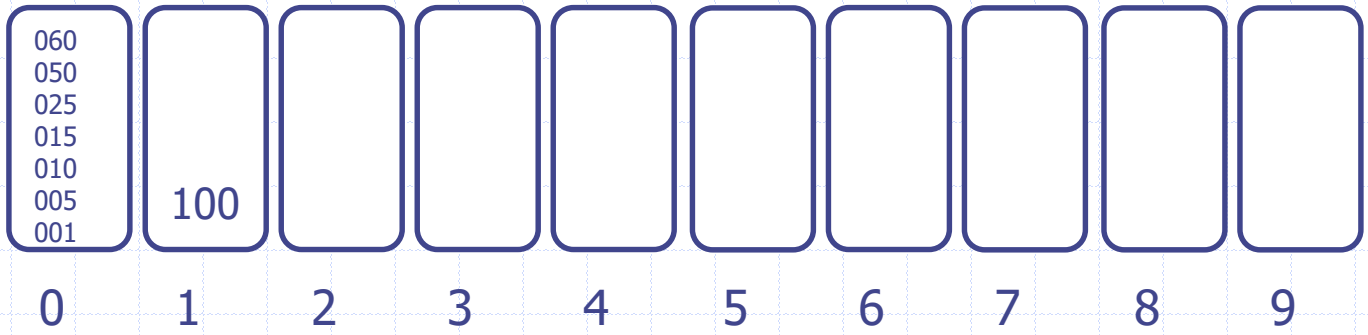

**100, 001, 005, 010, 015, 025, 050 ,060**

Pass 2 is completed

# Radix Sort – example

**100, 001, 005, 010, 015, 025, 050 ,060**

Sort numbers using the third digit :




**001, 005, 010, 015, 025, 050, 060, 100**

Pass 3 is completed

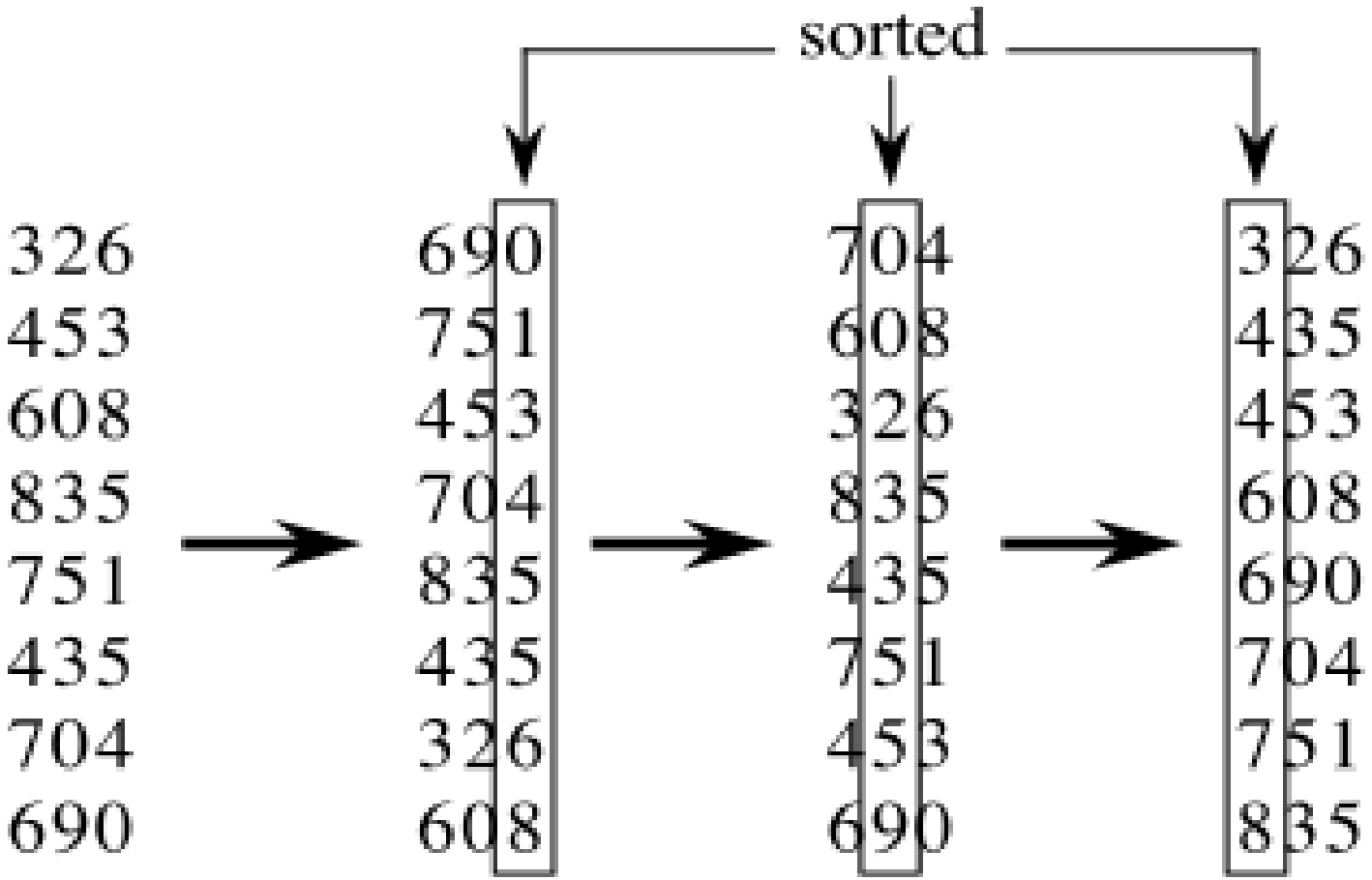
# Radix Sort – example

- stop after third pass
- remove the leading zeros :

  **1, 5, 10, 15, 25, 50, 60, 100**

  numbers are sorted !

# Radix Sort – example 2



# Radix Sort – running time

- Given n numbers of d digits each, where each digit may take up to k possible values, RADIX-SORT correctly sorts the numbers in $\boldsymbol{O(d\,(n+k))}$
- One pass of sorting per digit takes $\boldsymbol{O(n+k)}$ assuming that we use counting sort.
- There are d passes (for each digit)
- Assuming d=O(1) and k=O(n), running time is O(n)

# Choosing a Sorting Algorithm

- Running time (best, worst, and average).
- Implementation
- Stable or not
- Space complexity (in-place or not)
- Input size
- Format of the input list
- Nearly sorted list